



UNIVERSITY OF PISA

Master of Science in Computer Engineering

Electronics and Communications Systems

Project: Mini Router

Student:

Sebastiano Pala

Contents

1	Problem Description	3
1.1	Possible Applications	4
1.2	Possible Architectures	4
2	Implementation	5
2.1	VHDL Interface	5
2.2	Structure	6
2.2.1	MiniRouter	6
2.3	Decision Logic	7
2.4	VHDL implementation	8
2.4.1	Comparator	9
3	Verification	10
3.1	Testing	11
4	Synthesis	12
4.1	Timing Report	13
4.2	Most Critical Path	14
4.3	Power Consumption	15
4.4	Resource Utilization	15
5	Conclusion	16

Chapter 1

Problem Description

It is requested to design a synchronous mini-router that sends to the output a selection between two input sources.

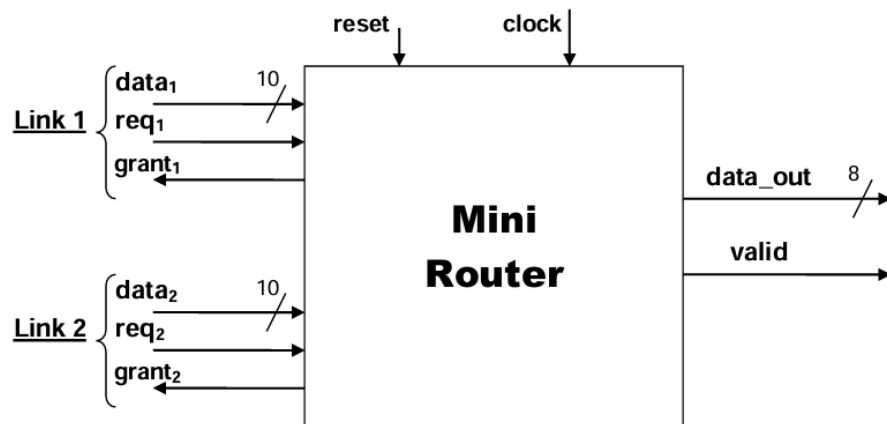


Figure 1.1: The router interface.

The interface of the circuit is characterized by two **links**, each with its own **data** lines, a **request** flag and a **grant** flag. The input data are represented on 8 bits, with 2 extra priority bits (0=minor priority, 3=maximum priority). The input data are valid when the corresponding req signal is set to 1. If in a clock cycle only one of the two req signals is high then the correspondent link's data are propagated to the output (**data_out**). Otherwise, if both of req signals are high, the mini-router has to select the link with the higher priority. If both links have the same priority (data conflict), then a single link is chosen following a Round Robin algorithm: after the reset, at the first conflict, link 1 is selected, at the second conflict the link 2, at the third conflict the link 1, and so on... The link selection and the possible conflict resolution is processed every clock cycle. When the output data are presented, the correspondent grant signal of the selected link is set high, as the **valid** output, for 1 clock cycle. The reset of the module is active high.

1.1 Possible Applications

The mini router is capable of managing both priority and fairness between two sources, and can be used in order to implement a simple Quality of Service mechanism. The module connects two input sources to the same output, and the priority mechanism guarantees that high priority inputs (interrupts, real time data) are forwarded before best effort traffic. The mini router can also be used to serialize two parallel input sources on a single output channel. The **valid** and **grant** signals manage the flow control, preventing data loss during link commutation.

1.2 Possible Architectures

The architecture chosen for the module's implementation is a fully combinatorial one, with the exception of the Round Robin bit and the output registers. This is not the only possible architecture, as the module could have been implemented as a finite state machine, with the following states:

- Idle
- Serving Link 1
- Serving Link 2

This approach though would have required a higher circuit complexity with little to no advantages.

Alternative Comparator

A comparator is necessary for the module as it allows it to compare the link priorities and forward the corresponding data. A possible architecture for this specific case (only a 2-bit comparator is needed) would be to implement the following logic operation:

$$\begin{aligned} equals &= (\overline{A_1 \oplus B_1}) \cdot (\overline{A_0 \oplus B_0}) \\ B_is_less &= ((A_1 \oplus A_0) \cdot (\overline{B_1})) + (A_1 \cdot A_0) \end{aligned}$$

In VHDL:

```

1    equals <= (A(1) xnor B(1)) and (A(0) xnor B(0));
2    B_is_less <= ((xor A) and (not B(1))) or (and A);

```

Although it may look like a better solution, it offers no flexibility and the efficiency gain compared to the current architecture (Ripple carry adder) is negligible. In addition adding even one bit to the priority requires a completely different circuit.

Chapter 2

Implementation

2.1 VHDL Interface

The interface is the following:

```
1 generic(  
2     DATA_SIZE : positive := 8;  
3     PRIORITY_SIZE : positive := 2  
4 );  
5 port(  
6     reset : in std_logic;  
7     clk : in std_logic;  
8  
9     -- LINK 1  
10    data1 : in std_logic_vector ((DATA_SIZE + PRIORITY_SIZE)-1  
11    downto 0);  
12    req1 : in std_logic;  
13    grant1 : out std_logic;  
14  
15    -- LINK 2  
16    data2 : in std_logic_vector ((DATA_SIZE + PRIORITY_SIZE)-1  
17    downto 0);  
18    req2 : in std_logic;  
19    grant2 : out std_logic;  
20  
21    data_out : out std_logic_vector (DATA_SIZE-1 downto 0);  
22    valid : out std_logic  
23 );
```

A wrapper has been used to wrap the module between two layers of registers. Necessary due to the fact that Vivado only performs timing analysis on Register - Logic - Register paths.

2.2 Structure

2.2.1 MiniRouter

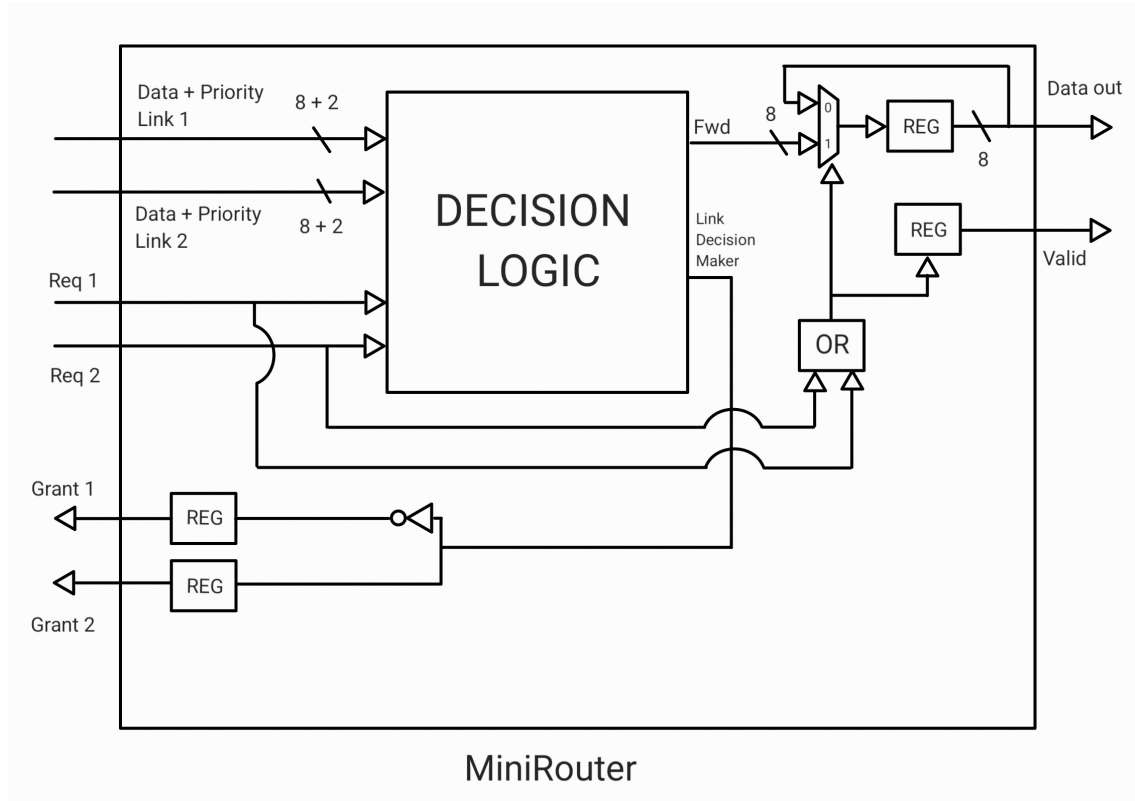


Figure 2.1: A simplified internal view of the circuit.

The MiniRouter is the main module and is a passive circuit, as it remains idle until one or both links request their data to be forwarded. A link can make a request by setting the **req** flag to 1, and the router will read the data at the next clock rising edge. When data has been forwarded, the router sets the **valid** flag and the **grant** flag in the respective link to 1, both for one clock cycle. The link selection and the possible conflict resolution is processed every clock cycle, and the **grant** outputs both depend on the result of the router's decision, represented through the **Link Decision Maker (LDM)** signal.

Working Conditions

In order to work properly, the circuit needs both **req** inputs to be driven, since the decision logic depends on both of them. Therefore if two sender modules are connected, they both have to drive their **req** output, otherwise the forwarding logic will be unpredictable.

Conflict Policy

The circuit can only forward data from one link at the time and, in order to solve conflicts, each link can specify the priority of their data alongside the data itself. A Comparator is used to figure out which link has the highest priority. If the priorities are equal, the module uses a Round Robin algorithm, implemented as follows: every time both links have the same priority, only one link is selected, and at the next conflict the other one is selected, and so on...

Data and Priority

Each message has the following structure: `[data,priority]` and is 10 bit long, composed of 8 bit for data and 2 bit for the priority of the message. We have in total 4 priority levels, going from **0** (lowest) to **3** (highest).

Reset

Despite being synchronous, the reset is asynchronous and active high. At reset, all outputs (**grant**, **req**, **data_out**, **valid**) are set to **0**, and the first link to be chosen in same priority conflicts is set to **Link 1**.

2.3 Decision Logic

The forwarding is implemented using a multiplexer, controlled by the signal **Link Decision Maker**. Link 1 is associated to $LDM = 0$, and Link 2 to $LDM = 1$. LDM is itself the output of another multiplexer, controlled by **Both Links Active** and **Equal Priorities**.

Case: One Link active

When only one link is active, **Link Decision Maker** = **req₂**. This is due to the fact that in this situation, **req₂** is equal to **0** when Link 1 is requesting, and **1** when Link 2 is requesting, thus the forwarded output will be the correct one.

Case: Different Priority

When both links are active with different priorities, the two are compared to determine which one is more important. When the equals signal is 0, the comparator returns 1 when Link 2 has a lower priority. In order to get the desired behavior ($LDM = 0$) LDM is set equal to **L2 Lower Priority**.

Case: Same Priority

When both links are active and they input the same priority, LDM will be equal to the output of Round Robin. When the previous condition is true, Round Robin will

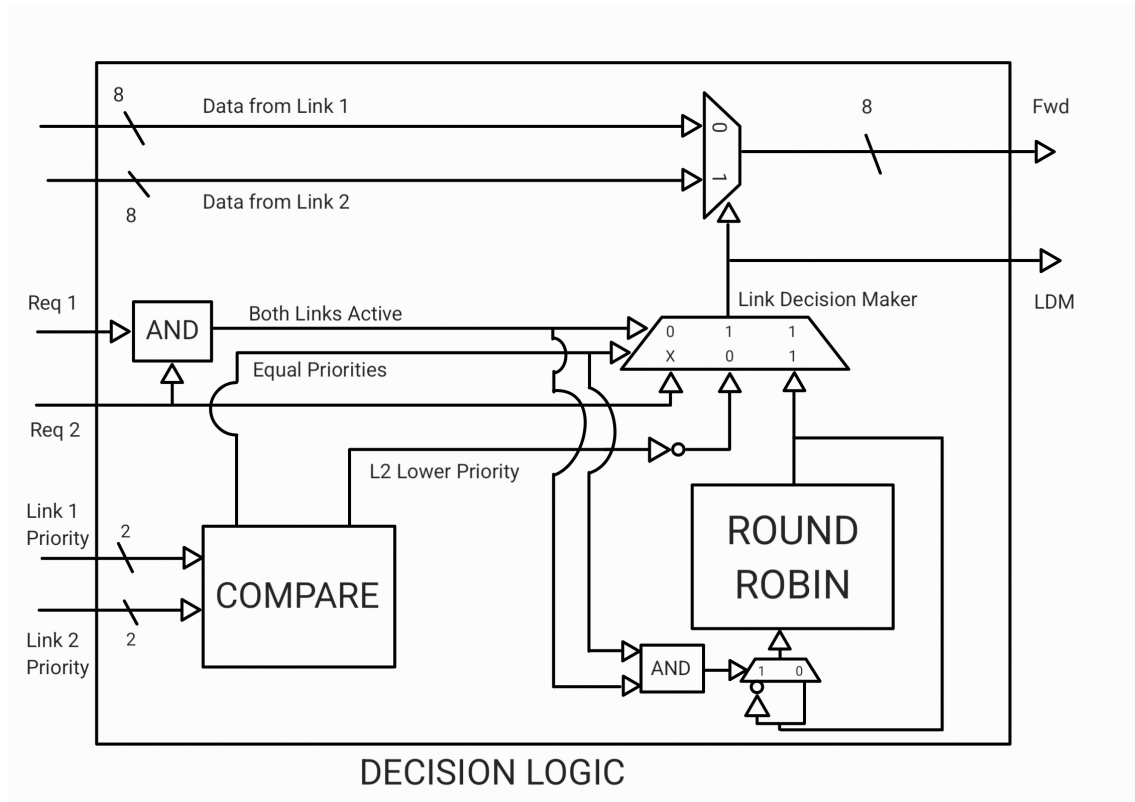


Figure 2.2: A closer look on how the module takes decisions.

also assume his previous value inverted at the next clock rising edge, thus switching every time the value is used. Round Robin starts at **0**, therefore Link 1 will be the first choice.

2.4 VHDL implementation

The main module uses a single process (**UPDATE_REG**) whose role consists in updating the values in the various registers at every clock rising edge. It handles both the normal functioning and the reset, which takes priority over the clock signal and is asynchronous.

```

1 UPDATE_REG: process(reset,clk)
2 begin
3     if(reset = '1') then -- async active high reset
4
5         data_out <= (others => '0');
6         valid <= '0';
7         round_robin_result <= '0';
8         grant1 <= '0';
9         grant2 <= '0';

```



```

10
11     elsif rising_edge(clk) then
12         if (req1 = '1' or req2 = '1') then -- update all
13
14             -- link_decision_maker = '0' -> we forwarded Link1
15             grant1 <= not link_decision_maker;
16
17             -- link_decision_maker = '1' -> we forwarded Link2
18             grant2 <= link_decision_maker;
19
20             valid <= '1';
21
22             round_robin_result <= next_round_robin;
23
24             data_out <= forwarded_data;
25
26         else -- if no link is active
27             grant1 <= '0';
28             grant2 <= '0';
29             valid <= '0';
30         end if;
31     end if;
32 end process;

```

All the other signals are combinatorial, and depend on the current input. The multiplexers used by the decision logic are implemented using a series of **WHEN ... ELSE** statements.

2.4.1 Comparator

The main module uses a comparator with the following interface:

```

1     generic(
2         SIZE : positive
3     );
4     port(
5         A : in std_logic_vector (SIZE-1 downto 0);
6         B : in std_logic_vector (SIZE-1 downto 0);
7
8         equals : out std_logic;
9         B_is_less : out std_logic
10    );

```

It's a simple wrapper for a Ripple Carry Adder, and its sole purpose is to invert B and set **Cin** = 1 in order to calculate the subtraction. The equals output is then calculated as a simple nor of the result, and B_is_less is the **Cout** of the RCA. Note that B_is_less = 1 whenever equals = 1, but it's not an issue as B_is_less is only used when equals = 0.

Chapter 3

Verification

The verification of the design was done using a VHDL testbench, simulated using ModelSim.

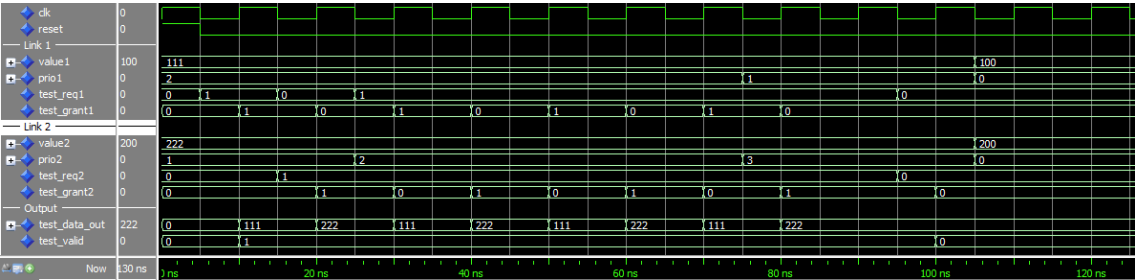


Figure 3.1: The testbench's waveform

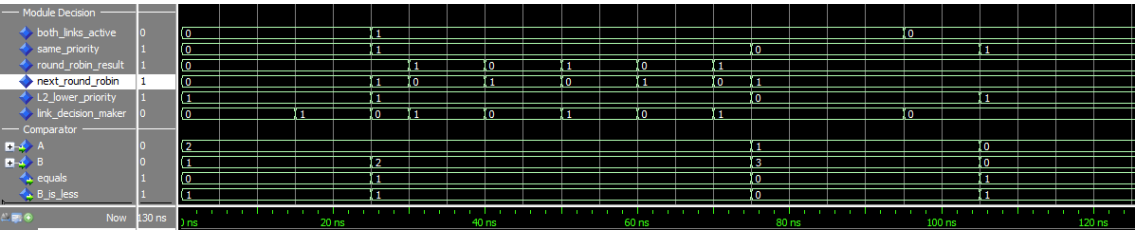


Figure 3.2: The decision values' waveform

The testbench is configured to validate the functionality of the MiniRouter by simulating two nodes requesting the forwarding of certain data, and checking the Router's output, confronting it with the expected output.

3.1 Testing

The testbench verifies the various use cases of the module through 7 tests, and if one fails Modelsim will interrupt the simulation.

The tests are the following:

1. Reset test: All outputs must be set to **0**
2. Single Link Forwarding: Only one link holds **req** to **1**
3. Switching Links: The active link and the inactive link switch roles, and the output must change accordingly
4. Round Robin: Both links are active with the same priority, therefore the router must apply the round robin algorithm
5. Highest Priority: Both links now have different priorities, hence the router must transmit only the highest priority data
6. No inputs: The **grant** and **valid** signals must be set to **0** when both links are off
7. No transparency: The module's output must be uninfluenced by variations in the data inputs when both links are off

The input given to the module has a half-clock-period delay, to improve the readability of the testbench's waveform.

The testbench will also print messages in the console showing which test was passed or failed.

```
VSIM 2> run -all
# ** Note: Test 1 PASSED: reset
#   Time: 5 ns   Iteration: 0   Instance: /minirouter_tb
# ** Note: Test 2 PASSED: single link forwarding
#   Time: 15 ns  Iteration: 0   Instance: /minirouter_tb
# ** Note: Test 3 PASSED: switching links
#   Time: 25 ns  Iteration: 0   Instance: /minirouter_tb
# ** Note: Test 4 PASSED: round robin
#   Time: 75 ns  Iteration: 0   Instance: /minirouter_tb
# ** Note: Test 5 PASSED: highest priority
#   Time: 95 ns  Iteration: 0   Instance: /minirouter_tb
# ** Note: Test 6 PASSED: no inputs
#   Time: 105 ns Iteration: 0   Instance: /minirouter_tb
# ** Note: Test 7 PASSED: no transparency
#   Time: 125 ns Iteration: 0   Instance: /minirouter_tb
# ** Note: ALL TESTS PASSED - Ending Simulation...
#   Time: 125 ns Iteration: 0   Instance: /minirouter_tb
```

Figure 3.3: The testbench's console output

Chapter 4

Synthesis

The synthesis was done with Vivado with the Zybo development board (xc7z010clg400-1). It was done without mapping the actual physical I/O ports of the FPGA pins, through the option "-mode out_of_context". Due to this fact, some warnings came up during simulation, and the accuracy of the timing is affected. However this does not impact the validity of the current results.

To obtain a timing report, a timing constraint was set at 125MHz.

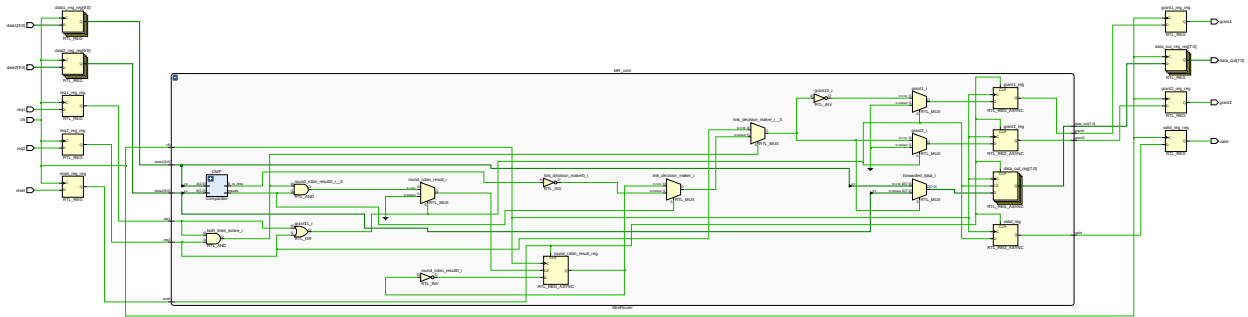


Figure 4.1: Vivado RTL schematic of the circuit.

4.1 Timing Report

The Design Timing Summary confirms that all user-specified constraints have been successfully met. A Worst Negative Slack (WNS) of 5.01 ns indicates that the clock period could be reduced by that amount without a setup violation occurring.

Since the constraint was 125MHz (8 ns clock period), this means that the circuit can have a clock frequency of up to $\frac{1}{T_{clk}-WNS} = \frac{1}{8ns-5.01ns} = \frac{1}{2.99ns} \simeq 334.448MHz$.

Similarly, the hold analysis reports a Worst Hold Slack (WHS) of 0.131 ns. Last but not least, the pulse width check has a worst pulse width slack of 3.500 ns, confirming that the clock pulses are wide enough for reliable register operation.

Design Timing Summary					
Setup		Hold		Pulse Width	
Worst Negative Slack (WNS): 5,010 ns		Worst Hold Slack (WHS): 0,131 ns		Worst Pulse Width Slack (WPWS): 3,500 ns	
Total Negative Slack (TNS): 0,000 ns		Total Hold Slack (THS): 0,000 ns		Total Pulse Width Negative Slack (TPWS): 0,000 ns	
Number of Failing Endpoints: 0		Number of Failing Endpoints: 0		Number of Failing Endpoints: 0	
Total Number of Endpoints: 43		Total Number of Endpoints: 43		Total Number of Endpoints: 46	
All user specified timing constraints are met.					

Figure 4.2: Vivado Timing Report.

4.2 Most Critical Path

The most critical path of this design goes from **round_robin_result_reg** to **data_out_reg[2]** and goes through two Look-Up Tables, responsible for the output value of both data and grant signals.

Intra-Clock Paths - 125MHz_constraint - Setup										
Name	Slack	Levels	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Clock Uncertainty
Path 1	5.010	2	10	MR_core/round_n_result_reg/C	MR_core/data_out_reg[2]/D	2.935	1.133	1.802	8.0	0.0
Path 2	5.013	2	10	MR_core/round_n_result_reg/C	MR_core/grant2_reg/D	2.984	1.133	1.851	8.0	0.0
Path 3	5.024	2	10	MR_core/round_n_result_reg/C	MR_core/data_out_reg[4]/D	2.923	1.133	1.790	8.0	0.0
Path 4	5.158	2	10	MR_core/round_n_result_reg/C	MR_core/data_out_reg[7]/D	2.789	1.133	1.656	8.0	0.0
Path 5	5.197	2	10	MR_core/round_n_result_reg/C	MR_core/data_out_reg[5]/D	2.750	1.133	1.617	8.0	0.0
Path 6	5.270	1	12	req1_reg/C	MR_core/data_out_reg[6]/D	2.677	0.642	2.035	8.0	0.0
Path 7	5.288	2	10	MR_core/round_n_result_reg/C	MR_core/data_out_reg[1]/D	2.657	1.133	1.524	8.0	0.0
Path 8	5.296	2	10	MR_core/round_n_result_reg/C	MR_core/data_out_reg[0]/D	2.652	1.133	1.519	8.0	0.0
Path 9	5.334	1	12	req2_reg/C	MR_core/data_out_reg[1]/CE	2.377	0.716	1.661	8.0	0.0
Path 10	5.334	1	12	req2_reg/C	MR_core/data_out_reg[6]/CE	2.377	0.716	1.661	8.0	0.0

Figure 4.3: Paths of the circuit.

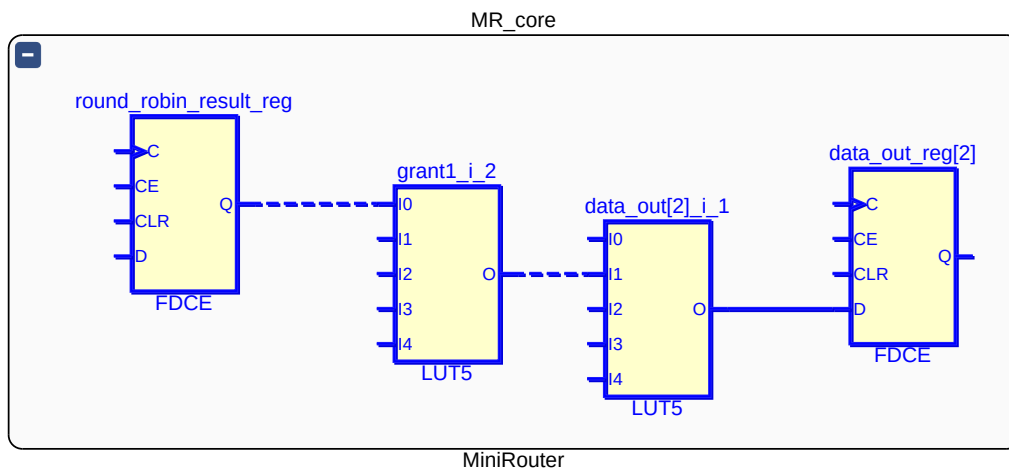


Figure 4.4: The critical path schematic.

4.3 Power Consumption

The report shows that the vast majority of the power (0.09W) is attributed to static device power, which is inherent to the FPGA hardware itself regardless of activity.

The remaining power (0.001W) corresponds to dynamic power, associated with the switching activity of the design. 84% of it is attributed to the clock signal, implying that the circuit uses a very efficient combinatorial logic.

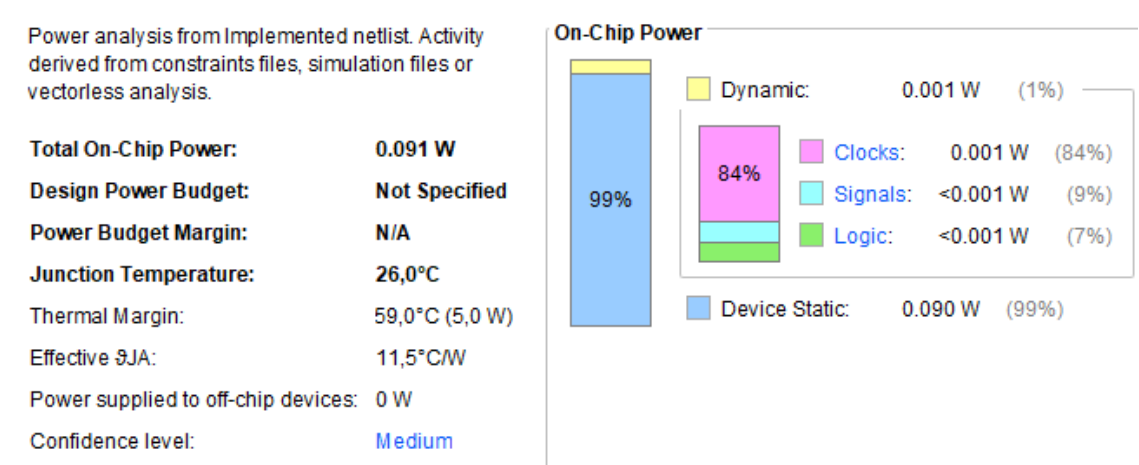


Figure 4.5: Power consumption report.

4.4 Resource Utilization

The implementation uses only 12 Look-Up Tables and 46 Flip-Flops and the utilization rate is respectively 0.07% and 0.13%. Such low percentages suggest that the design avoids excessive combinatorial complexity, and it can be incorporated in more complex systems without placing significant demands on the hardware.

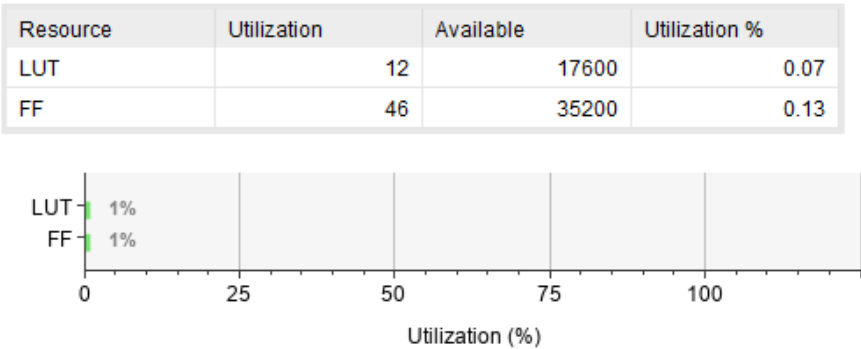


Figure 4.6: Vivado Utilization Report.

Chapter 5

Conclusion

In this project, it has been successfully described and implemented a Mini Router using VHDL, targeting an FPGA board. The main goal was to create a component capable of resolving data conflicts through a fixed priority setup and a Round Robin policy.

The current architecture for the decision logic is mostly combinatorial, making the circuit extremely efficient, especially when compared to a finite state machine implementation. This approach therefore minimized both resource usage and complexity.

The circuit utilizes only 12 Look-Up Tables and 46 Flip-Flops, allowing for integration into larger system. The timing analysis reports a maximum clock frequency of about 334 MHz, thanks to the logic being optimized for high-speed performance. The power analysis highlights the energy efficiency of the module, by showing that only 1% of the power consumption is attributed to the switching activity of the design.

The module's behavior was validated through a VHDL testbench, verifying both the routing logic's correctness and the reset behavior. The use of generic parameters also guarantees the design is scalable, allowing the usage of different data widths or priority levels without modifying the main logic.

Finally, for the synthesis and the implementation, a wrapper was used to obtain an accurate Register-to-Register timing analysis. The warnings generated during synthesis were due to the out-of-context mode and do not impact the validity of the internal logic or the reported results.